

XLSR-Mamba Field Report

Benchmarking a dual-column Mamba spoof detector on ASVspoof5, then fine-tuning it to catch Hindi, Bengali and Tamil voice deepfakes that survive WhatsApp-grade compression — and what that fine-tune cost the model's original skill.

Model XLSR-Mamba · 319.3M params **Hardware** RTX 5060 Laptop (8GB) · PyTorch 2.10 / cu128

Author tahermerchant2005

ASVSP00F5 BASELINE 2.77% EER · 9,000-clip dev subset · unmodified model	INDIC DEEPPKES, BEFORE → AFTER FINE-TUNE 46.9% → 0.31% EER on held-out hi/bn/ta test set · ~151× error reduction	COST: ASVSP00F5 AFTER FINE-TUNE 21.28% up from 2.77% — catastrophic forgetting, see §7
---	--	--

CONTENTS

01	The system under test	XLSR-Mamba architecture
02	ASVspoof5 baseline benchmark	EER 2.77%
03	The multilingual blind spot	EER 46.9%
04	Building the fine-tuning corpus	17,236 clips · 3 languages
05	Fine-tuning method	1.79M-param head
06	Results on Indic deepfakes	EER 0.31%
07	The forgetting cost	EER 21.28%

08 Limitations & honest caveats

09 Reproducing this report

`files & commands`

01 The system under test

A pure-PyTorch reconstruction of XLSR-Mamba (Xiao & Das, 2024, arXiv:2411.10027) — a dual-column bidirectional Mamba state-space model built on a frozen wav2vec 2.0 (XLSR) front end, published to detect synthetic and voice-converted speech.

01 XLSR-2.0 backbone — 317.5M params, frozen throughout this work

24-layer wav2vec 2.0 transformer pretrained on multilingual raw audio; extracts 1024-dim frame features at 16kHz / 320× downsampling (~62.5 frames/sec).

02 Front end — Linear(1024→144) + BatchNorm + SELU

Projects backbone features down to the 144-dim space the Mamba head operates in.

03 Dual-column Mamba conformer — 1.79M params, the part we train

Six bidirectional Mamba layers run the sequence forward and backward in parallel columns, each pooled by learned attention, concatenated, and classified into bonafide vs. spoof.

① **Why the split matters later:** because the backbone and front end never change, their output for a given clip is a fixed function of the waveform. Every clip's features can be computed once and cached — turning what would be hours of fine-tuning into single-digit minutes of head training. §5 relies on this.

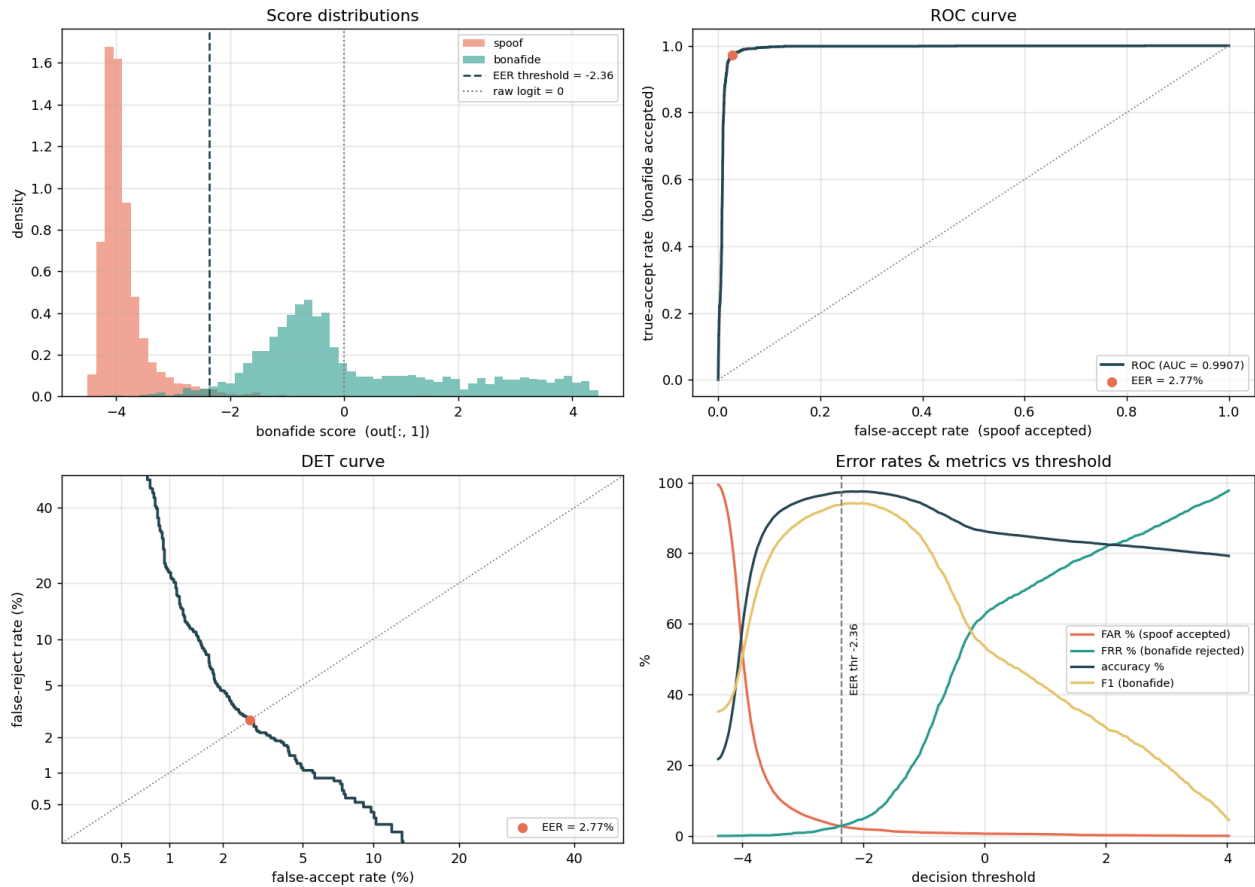
02 ASVspoof5 baseline benchmark

Before touching the model, we established its true out-of-the-box accuracy on the benchmark it was built for — correcting two bugs in the original evaluation script that had been silently loading half the trained weights.

⚠️ Bugs fixed before this number meant anything: the eval script hard-coded 6 Mamba layers per direction when the checkpoint holds 12, silently discarding 60 weight tensors while reporting "565/565 loaded OK" — and zero-padded short clips instead of repeat-tiling them, as the original training code does. Both are fixed; the script now hard-fails on any tensor mismatch instead of reporting a false pass.

METRIC	9,000-CLIP RUN	4,000-CLIP RUN
EER	2.77% (CI 2.39–3.17)	2.87% (CI 2.32–3.4)
ROC AUC	0.9907	0.98
Accuracy @ EER threshold	97.23%	97.1
F1 (bonafide)	93.72	93.
Sample	1,912 bonafide / 7,088 spoof	836 bonafide / 3,164 spo

XLSR-Mamba - ASVspoof5 dev track 1 (9000 utts: 1912 bonafide / 7088 spoof)

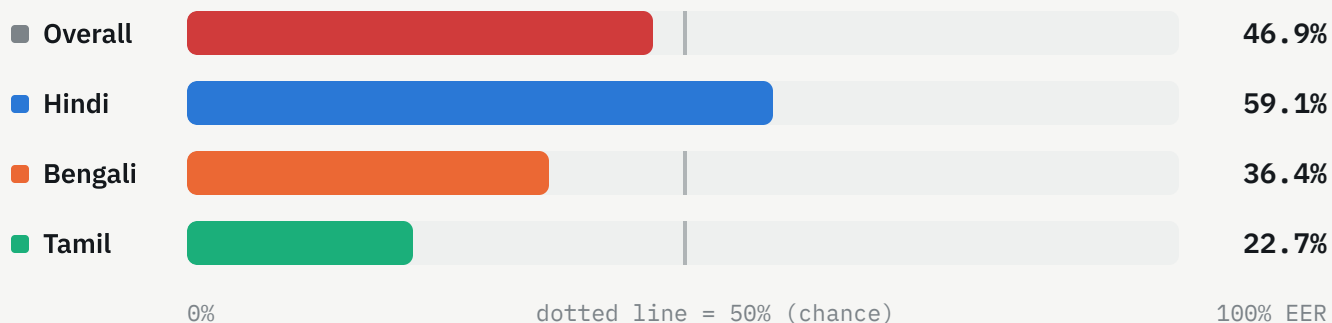


Score distributions (top-left) separate cleanly at -4 vs. 0 ; the DET curve (bottom-left) crosses the diagonal at $EER=2.77\%$; the raw-logit threshold of 0 (dotted line, top-left) is *not* where the model should be operated — see the naive-threshold pitfall below.

i A naive-threshold pitfall worth flagging: deciding "bonafide" at the raw $p \geq 0.5$ softmax cutoff gives only 86% accuracy on this benchmark — it catches nearly all spoofs but rejects most genuine speech, because the model's score distribution sits shifted negative. The EER threshold (-2.36) is the correct operating point and recovers 97.2% accuracy. The same trap reappears, worse, in §3.

03 The multilingual blind spot

The model above was trained on ASVspoof's English-centric, vocoder-focused attacks. Scored — unmodified — against modern Hindi/Bengali/Tamil TTS and voice-conversion deepfakes, it performs close to a coin flip:



Hindi is *worse than chance* — the model's scores are anti-correlated with the truth often enough to land above the 50% line. WhatsApp-style Opus compression barely moves any of these numbers (48.3% clean vs 46.1% compressed, overall), which says less about robustness and more about the fact that the model has no real signal to lose in the first place. This motivated everything that follows.

04 Building the fine-tuning corpus

Paired real and fake speech in three Indian languages, with a messaging-app compression pass baked directly into the data rather than left as an untested robustness claim.

SOURCE	ROLE	
IndicVoices-R (hi/bn/ta)	real · label 1	Studio-restored natural speech, 48
IndicSynth (hi/bn/ta)	fake · label 0	TTS & voice-conversion deepfakes (incl.

Every clip, resampled to 16kHz mono, then:

1. **Capped at 6 seconds**, centre-cropped.
2. **Split speaker-disjoint**, 80/10/10 train/dev/test — no speaker's voice appears in more than one split.
3. **Passed through a WhatsApp-equivalent codec**: Opus, 16kHz mono, ~24kbps, VoIP mode, encoded and decoded back with PyAV/libopus — the same lossy round-trip a WhatsApp voice note takes. Training clips are stored as *either* clean or compressed (p=0.5); dev/test clips are stored as *both*, so accuracy can be measured per condition.

LANGUAGE	REAL CLIPS	FAKE CLIPS
HI Hindi	3,889	4,263
BN Bengali	3,178	1,967
TA Tamil	2,019	1,920
Total	9,086	8,150

17,236 clips · ~2.9GB · train 12,730 / dev 1,904 / test 2,602

⚠ **Scaled down from a 6GB target.** HuggingFace's streaming API has no cheap resume — restarting mid-download re-streams and discards every already-seen row, which got expensive as cursors passed tens of thousands. Rather than pay repeated multi-hour re-catch-ups after transient stalls, the corpus was finalized at ~2.9GB. It was enough. The 12-language gated ai4bharat/IndicVoices set was also left out — gate approval didn't register for this account, and its audio format needs FFmpeg tooling not installed on this machine.

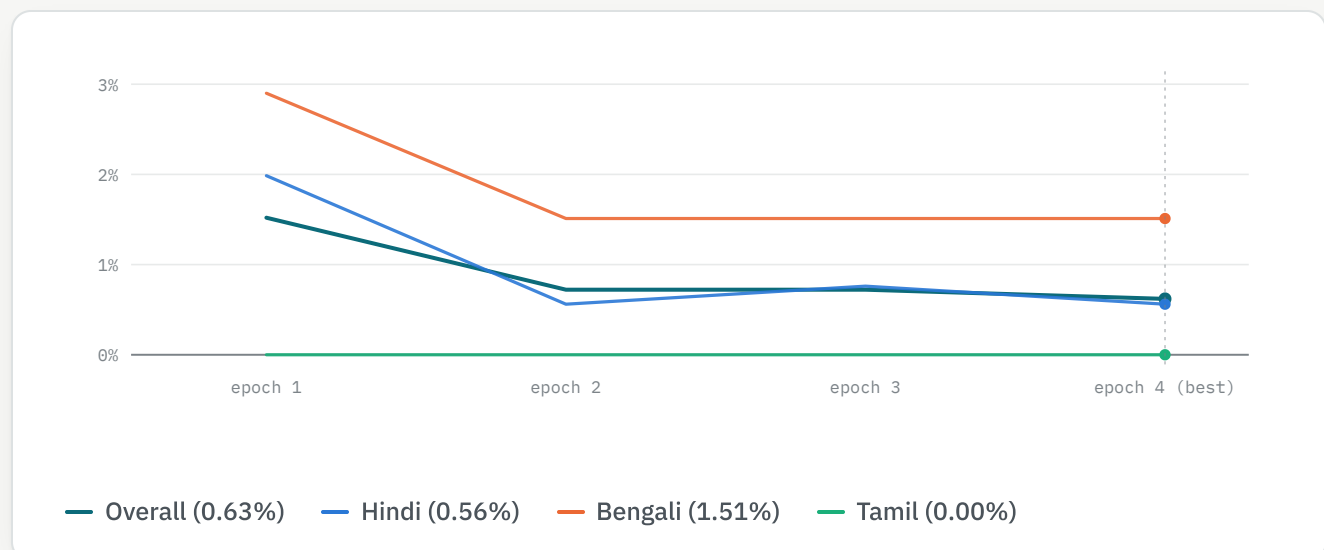
05 Fine-tuning method

Freeze everything except the 1.79M-param classification head, train it fast on cached features, and let a shrinking dev-set error rate decide when to stop.

SETTING	VALUE
Trainable params	1.79M of 319.3M (0.56%)
Optimizer	AdamW, lr 3e-4, cosine schedule
Loss	Cross-entropy, class-balanced by construction
Augmentation	Random 100-frame crop, frame masking, feature noise
Early stopping	Dev EER, patience 3
Batch size	64 (VRAM-bound on 8GB laptop GPU)

Sequence length was cropped to 100 of the cached 208 frames per training step: the dual-column Mamba scan backprops through a strictly sequential Python loop over every timestep, so its cost scales close to linearly with length — the crop roughly doubled training throughput at negligible accuracy cost, since a WhatsApp/vocoder artifact is present in any 2-second window, not only a full 4-second one.

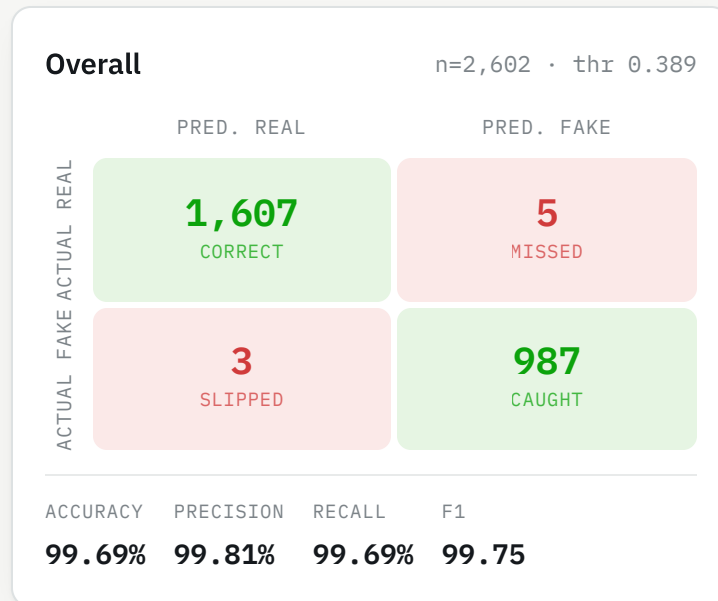
Training curve — dev-set EER by epoch



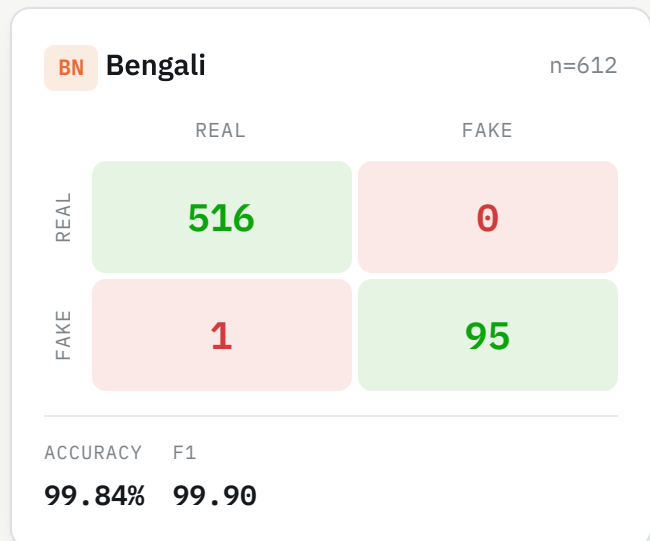
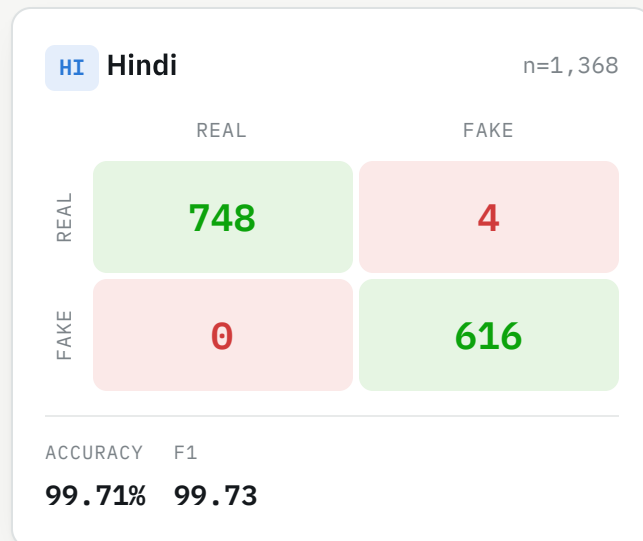
Training stopped after epoch 4 (best checkpoint kept) when the process crashed on a CUDA out-of-memory error at epoch 5 — VRAM ran within a few hundred MB of the 8GB ceiling for the whole run. See §8.

06 Results on Indic deepfakes

Scored on the held-out test set (2,602 clips, never seen in training) using the epoch-4 checkpoint. Overall EER: **0.31%**, down from 46.9% before fine-tuning.



By language



TA Tamil

n=622

	REAL	FAKE
REAL	343	1
FAKE	2	276

ACCURACY F1
99.52% 99.56

By compression condition**Clean audio**

n=1,301

	REAL	FAKE
REAL	803	3
FAKE	1	494

ACCURACY F1
99.69% 99.75

WhatsApp-compressed

n=1,301

	REAL	FAKE
REAL	804	2
FAKE	2	493

ACCURACY F1
99.69% 99.75

i The compression pass worked as intended: accuracy is identical, to two decimal places, on clean and WhatsApp-compressed audio (both 99.69%). Baking the codec into training rather than testing for it after the fact appears to have paid off directly.

07 The forgetting cost

The fine-tuned checkpoint was re-scored on the exact ASVspoof5 dev subset from §2, to check whether it kept its original skill.

BEFORE INDIC FINE-TUNE

2.77%

ASVspoof5 EER, same 9,000-clip subset

AFTER INDIC FINE-TUNE

21.28%

~7.7× higher error rate

This is a textbook case of **catastrophic forgetting**: retraining only the 1.79M-param head on a new, narrower distribution (three languages, a handful of TTS/VC systems) pulled the classifier's decision boundary away from the broad ASVspoof attack taxonomy it was originally tuned against — even though the 317.5M-param backbone underneath never moved. The head is small and fast to adapt, which is exactly why it is also fast to overwrite.

⚠ **This model is now a specialist, not a generalist.** Deploy `model_indic_ft.safetensors` only where the traffic is genuinely Hindi/Bengali/Tamil deepfake screening. For a detector that needs to hold both skills, the standard fixes are joint training (mix ASVspoof and Indic batches in the same fine-tuning run), a replay buffer of original-task examples, or an elastic-weight-consolidation penalty on the head — none of which this run attempted.

08 Limitations & honest caveats

- **Small dev/test sets per language.** Bengali dev is 454 clips, Tamil dev 258 — early-stopping decisions and the 0.00% Tamil EER numbers should be read as encouraging, not as statistically tight estimates.
- **Corpus scaled down from 6GB to ~2.9GB** mid-build (see §4) after streaming-resume costs made repeated interruptions expensive. More data, particularly more distinct TTS/VC systems, would likely improve robustness to attacks not represented here.
- **Only 12 languages' worth of "fake" generators are FreeVC24 and related IndicSynth models.** The fine-tuned head has not seen every deepfake technique in the wild — it has learned to catch the ones in this corpus extremely well.
- **Training crashed at epoch 5** on a CUDA out-of-memory error, likely triggered by a concurrent CPU-side evaluation job pushing the system over the edge while GPU VRAM was already pinned near its 8GB ceiling throughout. The epoch-4 checkpoint (already the best on dev EER) was kept as final rather than risk another crash chasing marginal gains.
- **Catastrophic forgetting on the original task** (§7) is real and unmitigated in this run.
- **The gated 12-language ai4bharat/IndicVoices dataset was not used** — access approval didn't register, and its audio needs FFmpeg tooling not present on this machine. This report covers 3 of India's 22 scheduled languages.

09 Reproducing this report

Every number above comes from a checked-in script and a file under `results/` or `results_indic/`.

FILE	
<code>evaluate_model.py</code>	ASVspoof5 baseline & forgetting-checkpoint
<code>build_indic_dataset.py</code>	Streams & compresses the Indic dataset
<code>cache_features.py</code>	Precomputes frozen-backbone features
<code>finetune_indic.py</code>	Trains the conformer head, logs finetune progress
<code>evaluate_indic.py</code> / <code>confusion_indic.py</code>	Per-language / per-codec EER & confusion matrix
<code>model_indic_ft.safetensors</code>	Final fine-tuned checkpoint

Full commands and narrative in `results_indic/README.md` and `results/RESULTS.md`.

